

Domanda **1**

Risposta errata

Punteggio ottenuto 0,00 su 3,00

Split in Mergesort

Si consideri l'implementazione delle liste semplici di interi:

```
struct listEl {
    int      info;
    struct listEl* next;
};

typedef struct listEl* list;
```

Si dia un'implementazione di costo $O(n)$ della funzione **split()**:

```
// post: modifica distruttivamente l in modo che l contenga circa la metà
//       degli elementi della lista data, e ritorna il puntatore alla
//       lista della seconda metà
list split(list l)
```

in modo tale che l'implementazione dell'algoritmo Mergesort seguente:

```
// post: ordina distruttivamente l in senso non decrescente
//       e ritorna il puntatore alla lista ordinata
list Mergesort(list l) {
    if (l == NULL || l->next == NULL)
        return l;
    else {
        list m = split(l);
        l = Mergesort(l);
        m = Mergesort(m);
        return merge(l, m);
    }
}
```

esegua correttamente.

Nota: nella specifica di **split()**, come in quella di **Mergesort()**, "distruttivamente" significa che NON devono essere allocati nuovi elementi, ma devono essere modificati soltanto i puntatori a next di quelli dati.

For example:

Test	Result
<pre>list l1 = Cons(4, Cons(1, Cons(3, Cons(9, Cons(1, NULL)))); printf("Lista data: "); printlist(l1); l1 = Mergesort(l1); printf("Output di Mergesort: "); printlist(l1);</pre>	<pre>Lista data: 4 1 3 9 1 Output di Mergesort: 1 1 3 4 9</pre>

Domanda **2**

Risposta errata

Punteggio ottenuto 0,00 su 4,00

Heapify per uno heap massimo

Pensando $H[0..hd]$ come uno heap, le funzioni left, right e parent

```
// pre: 0 <= i <= hd
// post: 2*i + 1 se <= hd, i altrimenti
int left(int i, int hd)

// pre: 0 <= i <= hd
// post: 2*i + 2 se <= hd, i altrimenti
int right(int i, int hd)

// pre: 0 <= i
// post: (i - 1) div 2 se i > 0, i altrimenti
int parent(int i)
```

hanno i significati:

- $left(i, hd)$ = indice del figlio sinistro di $H[i]$ se esiste nell'intervallo $0..hd$, i altrimenti
- $right(i, hd)$ = indice del figlio destro di $H[i]$ se esiste nell'intervallo $0..hd$, i altrimenti
- $parent(i)$ = indice del padre di $H[i]$ se esiste nell'intervallo $0..hd$, i altrimenti

Si implementi la funzione:

```
// pre: 0 <= i <= hd < dim H[], H[left(i, hd)..hd] e H[right(i, hd)..hd] sono heap massimi
// post: H[i..hd] è uno heap massimo
void maxHeapify(int H[], int i, int hd)
```

in modo che le funzioni preimplementate buildMaxHeap() e heapSort() eseguano correttamente.

Nota: per lo scambio di elementi su di un array si usi la funzione preimplementata swap():

```
// pre: 0 <= i, j, < dim A[]
// post: scambia A[i] con A[j]
void swap(int A[], int i, int j)
```

Domanda **3**

Risposta errata

Punteggio ottenuto 0,00 su 5,00

Predecessore in albero binario di ricerca

Si consideri la seguente implementazione degli alberi binari con chiavi intere e puntatore al padre:

```
struct BtreeNd {
    int         key;
    struct BtreeNd* parent;
    struct BtreeNd* left;
    struct BtreeNd* right;
};

typedef struct BtreeNd* btree;
```

dove per definizione il padre della radice è NULL. Si implmentino le seguenti funzioni:

```
// pre:  bt != NULL è un albero binario di ricerca
// post: ritorna il puntatore al nodo con chiave massima in bt
btree maxInBtree(btree bt)

// pre:  nd != NULL è un nodo di un albero binario di ricerca
// post: ritorna l'avo sinistro più prossimo a nd se esiste; NULL altrimenti
btree leftAncestor(btree nd)

// pre:  nd != NULL punta ad un nodo di un albero binario di ricerca
// post: ritorna il puntatore al predecessore di nd se esiste
//       NULL altrimenti
btree predecessor(btree nd)
```

dove in un albero di ricerca si definisce **avo sinistro** di un nodo nd un nodo il cui sottoalbero destro contenga nd (quindi è un avo di nd e si trova a "sinistra" di nd); si definisce **predecessore** di nd un nodo la cui chiave sia massima tra tutte quelle minori di nd->key.

Suggerimento: la funzione predecessor() utilizza le funzioni maxInBtree() e leftAncestor().

For example:

Test	Result
<pre>// test 1 btree bt1 = insert(45, NULL, NULL); bt1 = insert(23, bt1, NULL); bt1 = insert(52, bt1, NULL); bt1 = insert(12, bt1, NULL); bt1 = insert(48, bt1, NULL); bt1 = insert(1, bt1, NULL); bt1 = insert(7, bt1, NULL); printf("Albero dato:\n"); printtree(bt1, 0); predTest2(52, bt1); predTest2(48, bt1); predTest2(1, bt1);</pre>	<pre>Albero dato: 45 / \ 23 52 \ / \ 12 1 48 / \ 7 4 Predecessore di 52 = 48 Predecessore di 48 = 45 NON esiste il predecessore di 1</pre>